Faculty of Engineering
Cairo University
Credit Hour system spring 2024
Image Processing
Team #2

Maram Tarek	1200267
Sama Mohamed	1190582
Assem Sadek	1190541
Mohamed Mahmoud	1200438
Youssef Wael	4210227
Submitted to:	Eng, Omar Dawah

Eigen Faces

The purpose of using Eigenfaces in facial recognition is multifaceted and includes several key objectives:

Facial Recognition:

Primary Purpose, The main purpose of Eigenfaces is to enable efficient and accurate recognition of human faces from images or video frames. This involves identifying or verifying individuals based on their facial features.

Dimensionality Reduction:

Simplification: Eigenfaces reduce the high-dimensional data of facial images into a lower-dimensional subspace, simplifying the complexity of the data while preserving important features.

Efficiency: This reduction in dimensionality makes the recognition process faster and more computationally efficient.

Feature Extraction:

Principal Components: Eigenfaces help in extracting the most significant features from face images, capturing variations that are crucial for distinguishing different faces.

Pattern Recognition: By focusing on these key features, the method enhances the ability to recognize patterns and differences between faces.

Data Compression:

Storage Optimization: Eigenfaces allow for the compression of face data, reducing the amount of storage required for face images while retaining essential information.

Transmission: This is particularly useful for applications that require storing or transmitting large databases of face images.

Noise Reduction:

Signal Clarity: The use of PCA in Eigenfaces helps to filter out noise and irrelevant variations in the data, improving the clarity and quality of the facial features used for recognition.

Visualization:

Understanding: Eigenfaces provide a visual representation of the most significant features in face images, aiding in the understanding and analysis of what aspects are most important for recognition.

Debugging: This visualization can help in debugging and improving facial recognition systems by showing which features are being used.

Baseline for Advanced Techniques:

Foundation: Eigenfaces serve as a foundational technique in facial recognition, providing a starting point for more advanced methods and algorithms.

Comparison: They offer a benchmark for comparing the performance and efficiency of newer, more complex recognition algorithms.

Parameter entered:

1-TopK: This parameter specifies the number of top eigenfaces (principal components) to be displayed. Eigenfaces are the eigenvectors of the covariance matrix of the dataset, which represent the principal components of the face images.

2-ImageNum: This parameter specifies the index of the image from the dataset that the user wants to reconstruct using different numbers of principal components.

Comments on the Result:

Reading the Dataset: The Olivetti Faces dataset is loaded, and each image is flattened to create a matrix X, where each row represents a flattened image.

Mean Face Calculation: The mean face of the dataset is calculated and subtracted from each image to centre the data.

Covariance Matrix and Eigen Decomposition: The covariance matrix of the cantered data is computed, and its eigenvalues and eigenvectors are obtained. These eigenvectors represent the directions in which the data varies the most, and the corresponding eigenvalues represent the amount of variance in each direction.

Sorting and Selecting Eigenfaces: The eigenvalues and corresponding eigenvectors are sorted in descending order. The top k eigenfaces (principal components) are selected based on the topK parameter.

Variance Explained: The cumulative variance explained by the eigenvalues is calculated. This helps in understanding how much of the data's variability can be captured by the selected eigenfaces.

Image Reconstruction: For the specified image index (imageNum), the image is reconstructed using different numbers of principal components corresponding to variance percentages (10%, 15%, 30%, and 60%). This shows how the quality of the reconstructed image improves as more principal components are used.

Visualization: The top k eigenfaces are displayed, along with the original image and its reconstructions using the selected number of components. This visual representation helps in understanding the effect of using different numbers of eigenfaces on the reconstruction quality.

Part of code:

1. **Data Set:** Importing the Olivetti Faces dataset from Sklearn library. Storing each image flattened into a 1D array, in a matrix X where each row represents a flattened image.

```
from sklearn.datasets import fetch_olivetti_faces

# Read the Olivetti Faces dataset
faces = fetch_olivetti_faces()
images = faces.images

# Flatten the images to create a matrix where each row represents a flattened image
X = np.array([img.flatten() for img in images])
```

2. **Centring the data:** The mean face is calculated by averaging all the flattened images. Each image is then centered by subtracting the mean face, preparing the data for covariance matrix computation.

```
# Calculate the mean face
mean_face = np.mean(X, axis=0)

# Subtract the mean face from each image
X_centered = X - mean_face
```

3. **Eigen Decomposition:** Computing the covariance matrix, performing eigen decomposition, and sorting eigenfaces.

```
covariance_matrix = np.cov(X_centered, rowvar=False)

# Perform eigen decomposition
eigenvalues, eigenvectors = np.linalg.eigh(covariance_matrix)

# Sort eigenvectors by eigenvalues in descending order
sorted_indices = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[sorted_indices]
eigenvectors = eigenvectors[:, sorted_indices]
```

4. **Reconstructing the Image:** Selecting eigenfaces, reconstructing images with different components, and visualizing the results.

```
# Choose the top k eigenvectors (eigenfaces)
num_eigenfaces = 40
eigenfaces = eigenvectors[:, :num_eigenfaces]

# Calculate the cumulative variance explained by the eigenvalues
cumulative_variance = np.cumsum(eigenvalues) / np.sum(eigenvalues)

# Define variance percentages for reconstruction
variance_percentages = [0.10, 0.15, 0.30, 0.60]

# Find the number of components for each variance percentage
num_components = [np.argmax(cumulative_variance >= vp) + 1 for vp in variance_percentages]

# Reconstruct a sample image using the selected components
sample_image_index = 0  # Use the first image as an example
reconstructions = []

for n in num_components:
    eigenfaces_n = eigenvectors[:, :n]
    weights = np.dot(X_centered[sample_image_index], eigenfaces_n)
    reconstruction = np.dot(weights, eigenfaces_n.T) + mean_face
    reconstructions.append(reconstruction)
```

Output

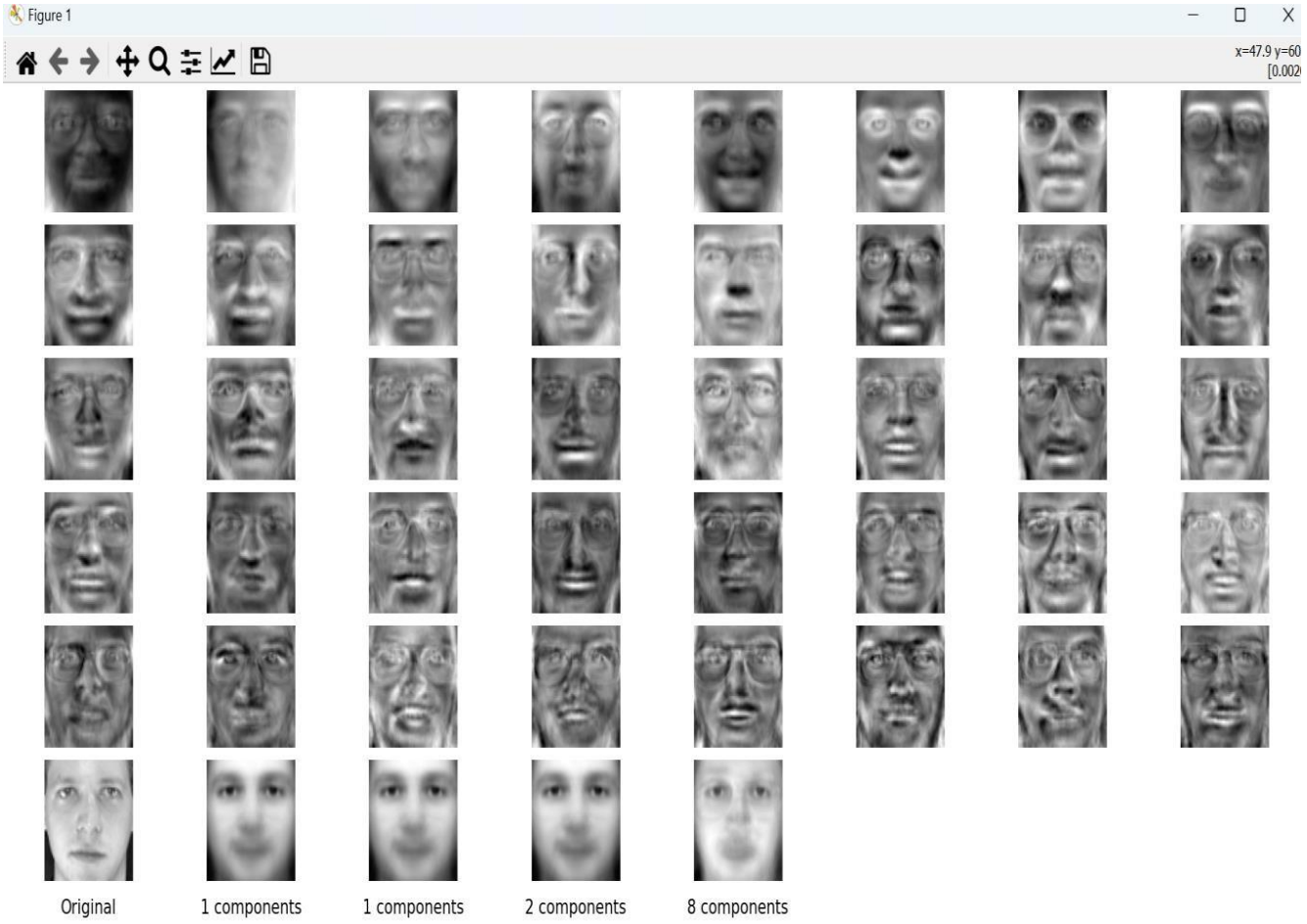
Figure 1
x=47.9 y=60
Original
1 components
1 components
2 components
8 components

Ellipse Detection

Ellipse detection has several benefits across various fields and applications:

Medical Imaging:

Diagnostics: Detecting elliptical shapes can help in identifying anatomical structures, such as blood vessels, cells, and other organs, which are often elliptical or circular in nature.
Tumor Detection: Ellipse detection algorithms can be used to identify and analyze tumors, which may have elliptical shapes.

Computer Vision:

Object Recognition: Ellipse detection is essential in recognizing objects that are circular or elliptical, like wheels, eyes, coins, and many other items.
Shape Analysis: It helps in analyzing and classifying shapes within an image, which is useful in various computer vision tasks.
Robotics:
Navigation: Robots can use ellipse detection to identify and navigate around circular objects or obstacles.
Grasping: Identifying the shape of objects helps robots in grasping and manipulating them effectively.

Entered parameter:

Image: The input image in which you want to detect ellipses.

k: The kernel size for the Gaussian blur, which helps in smoothing the image to reduce noise.

sigma: The standard deviation for the Gaussian blur, controlling the amount of blur.

a_low, a_high: The lower and upper bounds for the semi-major axis (a) of the ellipses to detect.

b_low, b_high: The lower and upper bounds for the semi-minor axis (b) of the ellipses to detect.

Expected Results:

Detected Ellipses:

Accuracy: The code should accurately detect and draw ellipses around features in the image that are elliptical in shape.
False Positives/Negatives: There should be a minimal number of false positives (incorrect ellipses) and false negatives (missed ellipses).

Visual Representation:

Ellipse Overlay: The detected ellipses should be clearly visible, overlaid on the original image with a red color (as specified in the code).
Observations and Comments:
Ellipses Detected:

Correct Detection: If the ellipses around expected features (e.g., eyes, wheels, circular objects) are detected accurately, the algorithm works well.
Missed Ellipses: If some elliptical features are not detected, consider adjusting the parameters for the semi-major and semi-minor axes ranges (a_range and b_range), or the edge detection thresholds.
False Positives: If there are many incorrect ellipses, this may indicate that the threshold for voting (set at 50% of the max accumulator value) is too low. Increasing this threshold might reduce false positives.

Visual Clarity:

Ellipse Size and Scaling: The ellipses are scaled down by a factor of 0.1 in the code. This scaling might make the ellipses too small. Adjust the scale_factor to ensure the ellipses are appropriately sized.

Performance:

Computation Time: The voting process can be time-consuming, especially for large images or those with many edges. If the computation takes too long, consider optimizing the loops or implementing parallel processing.
Sample Result Analysis:

Detected Ellipses: Ellipses are detected around some features, but some expected ellipses are missing.
False Positives: A few ellipses are drawn where there are no clear elliptical features.
Ellipse Size: The ellipses appear smaller than expected due to the scaling factor.

Threshold Adjustment:

Increase the threshold to reduce false positives:
threshold = max(accumulator.values()) * 0.6
# Increase threshold from 0.5 to 0.6.

Part of code

1. **Parameter initialization &accumulator setup:**
   After the image upload and converting to grayscale and applying the gaussian blur we.
   - Set Parameter Ranges: Define the ranges for semi-major axis (a), semi-minor axis (b), and angle (theta).
   - Initialize Accumulator: Create an empty dictionary to accumulate votes for potential ellipse parameters.

```
# Parameters
a_range = (a_low, a_high)
b_range = (b_low, b_high)
theta_range = np.deg2rad(np.arange(0, 90))  # Convert theta to radians
print("Parameters set.")

# Initialize the accumulator as a dictionary
accumulator = {}
print("Accumulator initialized.")
```

2. **Voting Process:** Iterate through edge points for each edge point detected by the Canny edge detector, vote for potential ellipse parameters in the accumulator.

```
# Voting
for y, x in np.argwhere(edges):
    for a in range(*a_range):
        for b in range(*b_range):
            for theta in theta_range:
                x_c = int(round(x - a * np.cos(theta)))
                y_c = int(round(y + b * np.sin(theta)))
                if 0 <= x_c < image.shape[1] and 0 <= y_c < image.shape[0]:
                    key = (y_c, x_c, a, b, theta)
                    if key in accumulator:
                        accumulator[key] += 1
                    else:
                        accumulator[key] = 1
print("Voting completed.")
```

3. **Identify and draw potential ellipse**:
    - Potential Ellipse: Identify the ellipse parameters that have received votes exceeding a certain threshold.
    - Draw Ellipse: Draw the detected ellipses on the original image.

```
# Find potential ellipses
threshold = max(accumulator.values()) * 0.5
potential_ellipses = [k for k, v in accumulator.items() if v >= threshold]
print(f"Potential ellipses identified: {len(potential_ellipses)}")

# Ellipse Fitting
scale_factor = 0.1  # Further reduce the size of the ellipse
for y_c, x_c, a, b, theta in potential_ellipses:
    scaled_a = int(a * scale_factor)
    scaled_b = int(b * scale_factor)
    cv2.ellipse(image, (x_c, y_c), (scaled_a, scaled_b),
                np.degrees(theta), 0, 360, (0, 0, 255), 1)
print("Ellipses drawn on image.")

return (image)
```

Output